

WEEK-1,2 LAB PDF

Question

Fundamentals of Object Oriented Programming: Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like

deposits, withdrawals, and transfers.

Simple Steps (Lab Record Format)

1. Start.
2. Create the account class with account number, balance and account type.
3. Implement deposit, withdraw, transfer and display methods.
4. Create the savingaccount class by inheriting account.
5. Override the interest calculation method.
6. Create the currentaccount class by inheriting account.
7. Override the withdraw method to support overdraft.
8. Create Savings and Current Account objects in the main method.
9. Perform deposit, withdrawal and interest calculation.
10. Transfer money between accounts.
11. Display the final account details.
12. Stop.

Algorithm: Bank Management System

Step 1: Start

Step 2: Create the account class

- Declare variables:
 - o accno o
 - balance o
 - acctype
- Create a constructor to initialize account details.

Step 3: Define methods in account

- deposit(amount)
 - o Add the amount to balance.
 - o Display the updated balance.
- withdraw(amount)
 - o If balance is sufficient:
 - Subtract amount from balance.
 - Display the updated balance.
 - o Otherwise display "Insufficient Balance".
- transfer(receiver, amount)
 - o Check whether sender has sufficient balance.
 - o Deduct amount from sender.
 - o Add amount to receiver.
 - o Display receiver's updated balance.
- calinterest()
 - o Empty method (to be overridden).
- display()
 - o Display account number, account type and balance.

Step 4: Create the savingaccount class

- Inherit the account class.
- Add variable interest.
- Create constructor using super().
- Override calinterest().
 - Calculate:
 - $\text{interest} = \text{interest} \times \text{balance} / 100$
 - Display the calculated interest.

Step 5: Create the currentaccount class

- Inherit the account class.
- Add variable overdraft.
- Create constructor using super().
- Override withdraw().
 - If withdrawal amount $\leq$ balance + overdraft:
 - Withdraw amount.
 - Display success message.
 - Otherwise display "Overdraft Limit Exceeded".

Step 6: Create the BankManagement class

- Create one Savings Account object.
- Create one Current Account object.

Step 7: Perform Savings Account operations

- Display account details.
- Deposit money.
- Withdraw money.
- Calculate interest.

Step 8: Perform Current Account operations

- Display account details.
- Deposit money.

- Withdraw money using overdraft facility.

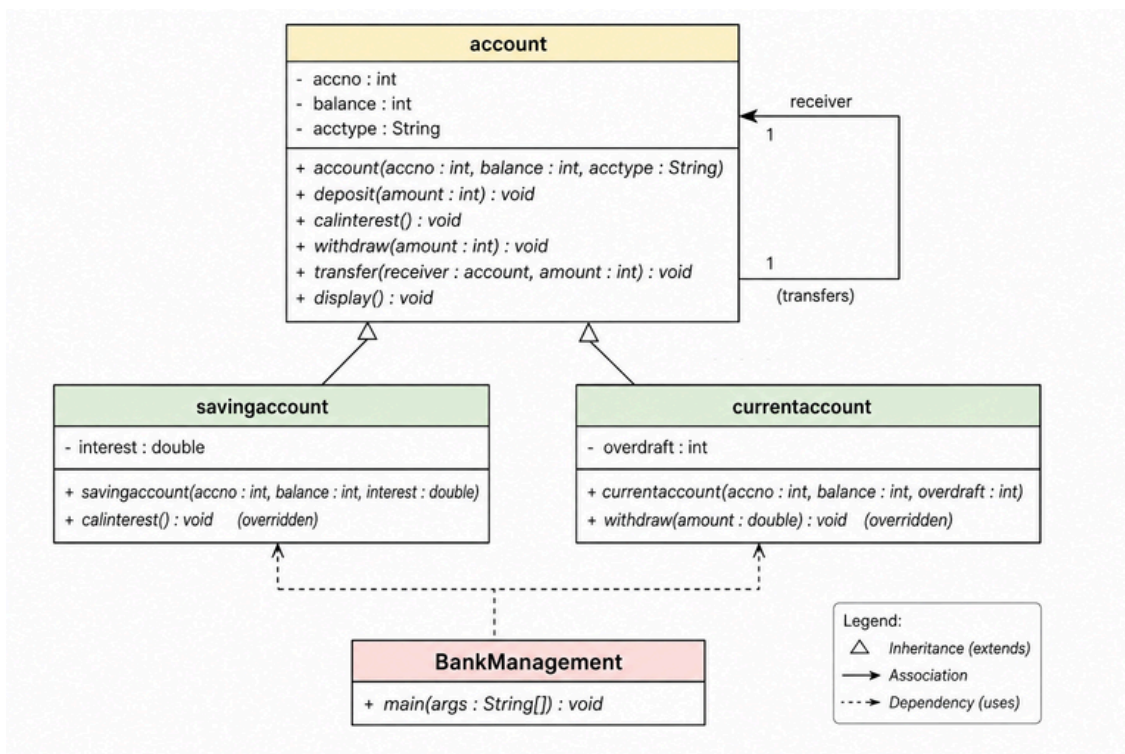
Step 9: Transfer money

- Transfer money from Savings Account to Current Account.

Step 10: Display final account details

Step 11: Stop

UML DIAGRAM



Code

```
class account{
    int accno;
    int balance;
    String acctype;
    public account(int accno,int balance,String acctype){
        this.accno=accno;
        this.balance=balance;
        this.acctype=acctype;
    }
    void deposit(int amount){
        balance=balance+amount;
        System.out.println("balance after deposit is:"+balance);
    }
    void calinterest(){
    }
    void withdraw(int amount){
        if (balance>=amount) {
            balance=balance-amount;
            System.out.println("balance after withdrawal is :"+balance);
        }
        else
            System.out.println("insufficient balance");
    }
    void transfer(account recevier,int amount){
        if (balance>=amount) {
            balance=balance-amount;
            recevier.balance=recevier.balance+amount;

            System.out.println("balance after transfer is :"+recevier.balance);
        }
        else
            System.out.println("insufficient balance");
    }
    void display(){
        System.out.println(accno);
        System.out.println(acctype);
        System.out.println(balance);
    }
}
class savingaccount extends account{
    double interest;
    public savingaccount(int accno,int balance,double interest){
        super(accno, balance, "saving");
        this.interest=interest;
    }
    @Override
    void calinterest(){
        interest=interest*balance/100;
        System.out.println("interest is:"+interest);
    }
}
class currentaccount extends account{
    int overdraft;

    public currentaccount(int accno,int balance, int overdraft){
        this.overdraft=overdraft;
        super(accno, balance,"current account");
    }
    public void withdraw(double amount) {
        if (amount <= balance + overdraft) {
            balance -= amount;
            System.out.println("Amount Withdrawn: " + amount);
        } else {
            System.out.println("Overdraft Limit Exceeded");
        }
    }
}
```

```

    public class BankManagement {
    public static void main(String[] args) {

        savingaccount s1 = new savingaccount(101,20000,5);
        currentaccount c1 = new currentaccount(102,15000,5000);

        System.out.println("Savings Account");
        s1.display();

        s1.deposit(5000);
        s1.withdraw(3000);
        s1.calinterest();

        System.out.println("\nCurrent Account");
        c1.display();

        c1.deposit(2000);
        c1.withdraw(18000);

        System.out.println("\nTransfer Money");
        s1.transfer(c1,4000);

        System.out.println("\nFinal Details");

        s1.display();
        c1.display();
    }
}

```

Sample Output

```

Savings Account
101
saving
20000
balance after deposit is:25000
balance after withdrawal is :22000
interest is:1100.0

```

```

Current Account
102
current account
15000
balance after deposit is:17000
Amount Withdrawn: 18000

```

```

Transfer Money
balance after transfer is :3000

```

```

Final Details
-----
101
saving
18000

```

```

102
current account

```

RESULT

The Bank Management System was successfully implemented and executed.

The program successfully performed deposit, withdrawal, interest calculation, and money transfer operations for savings and current accounts.

The final account details were displayed successfully.